



Orbs PerpsRollup Security Audit Report

May 27, 2026



Contents

1 Introduction

[1.1 About Orbs PerpsRollup](#)

[1.2 Source Code](#)

[1.3 Revision History](#)

2 Overall Assessment

3 Vulnerability Summary

[3.1 Overview](#)

[3.2 Security Level Reference](#)

[3.3 Vulnerability Details](#)

4 Appendix

[4.1 About AstraSec](#)

[4.2 Disclaimer](#)

[4.3 Contact](#)

1 Introduction

1.1 About Orbs PerpsRollup

PerpsRollup is an upgradeable on-chain coordinator that records and advances a rollup state root based on authorized off-chain signatures. It holds ERC-20 deposits, executes signed state changes (including withdrawals), and enforces role-based permissions for signers and upgrades.



1.2 Source Code

The audit is conducted based on the deployed ERC1967Proxy at <https://arbiscan.io/address/0x45446ce74d6f3902ec37fef2f38b046e327bf1c2> and its Implementation at <https://arbiscan.io/address/0x344ce08dd8becae5e1f29c7697e9424bceaf6b03>.

Files in scope: PerpsRollup.sol, PerpsEvents.sol.

1.3 Revision History

Version	Date	Description
v1.0	May 27, 2026	Initial Audit

2 Overall Assessment

This report has been compiled to identify issues and vulnerabilities within the Orbs PerpsRollup contract. Throughout this audit, we identified a total of 2 issues spanning various severity levels. By employing auxiliary tool techniques to supplement our thorough manual code review, we have discovered the following findings.

Severity	Count	Acknowledged	Won't Do	Addressed
Critical	—	—	—	—
High	—	—	—	—
Medium	1	1	—	—
Low	—	—	—	—
Informational	1	—	1	—

3 Vulnerability Summary

3.1 Overview

Click on an issue to jump to it, or scroll down to see them all.

M-1

[Potential Risks Associated with Centralization](#)

I-1

[Recommended Checks-Effects-Interactions \(CEI\) Pattern](#)

3.2 Security Level Reference

In web3 smart contract audits, vulnerabilities are typically classified into different severity levels based on the potential impact they can have on the security and functionality of the contract. Here are the definitions for critical-severity, high-severity, medium-severity, and low-severity vulnerabilities:

Severity	Acknowledged
C-X (Critical)	A severe security flaw with immediate and significant negative consequences. It poses high risks, such as unauthorized access, financial losses, or complete disruption of functionality. Requires immediate attention and remediation.
H-X (High)	Significant security issues that can lead to substantial risks. Although not as severe as critical vulnerabilities, they can still result in unauthorized access, manipulation of contract state, or financial losses. Prompt remediation is necessary.
M-X (Medium)	Moderately impactful security weaknesses that require attention and remediation. They may lead to limited unauthorized access, minor financial losses, or potential disruptions to functionality.
L-X (Low)	Minor security issues with limited impact. While they may not pose significant risks, it is still recommended to address them to maintain a robust and secure smart contract.
I-X (Informational)	Warnings and things to keep in mind when operating the protocol. No immediate action required.
U-X (Undetermined)	Identified security flaw requiring further investigation. Severity and impact need to be determined. Additional assessment and analysis are necessary.

3.3 Vulnerability Details

3.3.1 [M-1] Potential Risks Associated with Centralization

Target	Category	IMPACT	LIKELIHOOD	STATUS
PerpsRollup.sol	Centralization	Medium	Medium	Acknowledged

The PerpsRollup contract vests significant protocol control in privileged roles, namely `ROLLUP_MANAGER_ROLE` and `DEFAULT_ADMIN_ROLE`. `ROLLUP_MANAGER_ROLE` holders validate EIP-712 TEE operations that advance the rollup state (including processing deposits and withdrawals), while `DEFAULT_ADMIN_ROLE` can manage those roles and authorize upgrades via `_authorizeUpgrade()`.

On-chain data shows that `ROLLUP_MANAGER_ROLE` is held by `0x4e42...29DA` and `DEFAULT_ADMIN_ROLE` is held by `0x8555...F925`; both are externally-owned accounts (EOAs). Concentrating these powers in a few entities introduces centralization risk: a compromised or malicious signer/admin could issue fraudulent state changes, withdraw assets, or alter role/upgrade governance.

```
arbitrum_0x45446ce74d6f3902ec37fef2f38b046e327bf1c2_code - PerpsRollup.sol
143 bytes32 structHash = keccak256(...);
144 bytes32 digest = _hashTypedDataV4(structHash);
145 address signer = ECDSA.recover(digest, op.teeSignature);
146 require(hasRole(ROLLUP_MANAGER_ROLE, signer), "invalid signer");
147
148 // Side effects
149 if (op.eventSignature == DEPOSIT_EVENT_SIGNATURE) {
150     bytes32 depositId = keccak256(op.logData);
151     require($.pendingDeposits[depositId], "deposit not pending");
152     $.pendingDeposits[depositId] = false;
153 } else if (op.eventSignature == WITHDRAWAL_EVENT_SIGNATURE) {
154     (, address recipient, uint256 amount) = abi.decode(op.logData, (bytes32, address, uint256));
155     IERC20($.asset).safeTransfer(recipient, amount);
156 }
```

Remediation To mitigate the identified issue, it is recommended to introduce multi-sig mechanism to undertake the role of the privileged accounts. Moreover, it is advisable to implement timelocks to govern all modifications to the privileged operations.

Response By Team This issue has been acknowledged by the team.

3.3.2 [I-1] Recommended Checks-Effects-Interactions (CEI) Pattern

TARGET	CATEGORY	IMPACT	LIKELIHOOD	STATUS
PerpsRollup.sol	Coding Practice	NA	NA	Acknowledged

`updateStateRoot()` performs external ERC-20 transfers ([IERC20.safeTransfer](#)) while iterating signed operations without following the Checks -> Effects -> Interactions pattern. A token callback (or a malicious/atypical token) could re-enter and modify storage relied upon later in the same execution. Although the current asset is USDC (no known reentrancy issue), we recommend enforcing CEI: apply all checks and write state updates before any external calls, or collect withdrawals and perform transfers after state updates (batch transfers). As defense-in-depth, add explicit reentrancy protection (e.g., `nonReentrant` guard).

```
arbitrum_0x45446ce74d6f3902ec37fef2f38b046e327bf1c2_code - PerpsRollup.sol

124 function updateStateRoot(SignedOperation[] calldata operations) external {
125     V1Storage storage $ = _getV1Storage();
126     require(operations.length > 0, "empty");
127
128     uint256 expectedSeq = $.sequenceNumber;
129     bytes32 expectedPrevRoot = $.stateRoot;
130
131     for (uint256 i = 0; i < operations.length; i++) {
132         ...
133         bytes32 digest = _hashTypedDataV4(structHash);
134         address signer = ECDSA.recover(digest, op.teeSignature);
135         require(hasRole(ROLLUP_MANAGER_ROLE, signer), "invalid signer");
136
137         // Side effects
138         if (op.eventSignature == DEPOSIT_EVENT_SIGNATURE) {
139             bytes32 depositId = keccak256(op.logData);
140             require($.pendingDeposits[depositId], "deposit not pending");
141             $.pendingDeposits[depositId] = false;
142         } else if (op.eventSignature == WITHDRAWAL_EVENT_SIGNATURE) {
143             (, address recipient, uint256 amount) = abi.decode(op.logData, (bytes32, address, uint256));
144             IERC20($.asset).safeTransfer(recipient, amount);
145         }
146     }
147
148     // Update state to last operation's values
149     $.stateRoot = expectedPrevRoot;
150     $.sequenceNumber = expectedSeq;
151 }
```

Remediation Refactor `updateStateRoot()` to follow Checks-Effects-Interactions (CEI) pattern or add `nonReentrant` guard.

4 Appendix

4.1 About AstraSec

AstraSec is a blockchain security company that serves to provide high-quality auditing services for blockchain-based protocols. With a team of blockchain specialists, AstraSec maintains a strong commitment to excellence and client satisfaction. The audit team members have extensive audit experience for various famous DeFi projects. AstraSec's comprehensive approach and deep blockchain understanding make it a trusted partner for the clients.

4.2 Disclaimer

The information provided in this audit report is for reference only and does not constitute any legal, financial, or investment advice. Any views, suggestions, or conclusions in the audit report are based on the limited information and conditions obtained during the audit process and may be subject to unknown risks and uncertainties. While we make every effort to ensure the accuracy and completeness of the audit report, we are not responsible for any errors or omissions in the report.

We recommend users to carefully consider the information in the audit report based on their own independent judgment and professional advice before making any decisions. We are not responsible for the consequences of the use of the audit report, including but not limited to any losses or damages resulting from reliance on the audit report.

This audit report is for reference only and should not be considered a substitute for legal documents or contracts.

4.3 Contact

Phone	+86 156 0639 2692
Email	contact@astrasec.ai
Twitter	https://twitter.com/AstraSecAI